РЕФЕРАТ

Расчётно-пояснительная записка содержит 8 рисунков, 17 таблиц, 10 источников.

Ключевые слова: база данных, мировой чемпионат, гоночный заезд, результат гонки, квалификация, экипаж, гоночный класс, зачётные очки, статистика участников, PostgreSQL, Redis, конкурентная нагрузка.

Объектом разработки является база данных для хранения результатов гоночных заездов мировых чемпионатов и сведений об участниках соревнований.

Цель работы — спроектировать и реализовать базу данных, позволяющую учитывать результаты квалификаций и гонок, рассчитывать зачётные очки и получать статистику гонщиков и команд.

В ходе работы проанализированы предметная область и существующие источники гоночных результатов, сформулированы требования и ограничения, выбрана модель хранения данных. Спроектированы структура базы данных, ограничения целостности, ролевая модель и способы доступа. Реализованы сущности и расчётные функции PostgreSQL, описана схема внешнего кэширования в Redis. Проверена корректность расчётов и исследована зависимость времени выполнения запросов от объёма данных и конкурентной нагрузки.

СОДЕРЖАНИЕ

РЕФЕРАТ **4**

ВВЕДЕНИЕ **6**

1 Аналитический раздел **7**

1.1 Анализ существующих решений 7

1.2 Уточнение требований к системе 8

1.3 Ограничения предметной области 9

1.4 Анализ баз данных по модели хранения данных 10

2 Конструкторский раздел **12**

2.1 Проектирование базы данных 12

2.2 Ограничения целостности 14

2.3 Проверка соответствия третьей нормальной форме 14

2.4 Проектирование ролевой модели базы данных 15

2.5 Проектирование способов взаимодействия с базой данных 16

2.6 Проектирование функций расчёта очков и статистики 17

2.7 Индексы и согласованность операций 21

3 Технологический раздел **22**

3.1 Средства реализации 22

3.2 Реализация сущностей базы данных 22

3.3 Реализация функций базы данных 23

3.4 Роли и привилегии 24

3.5 Приложение доступа к базе данных 25

3.6 Внешнее кэширование с использованием Redis 26

3.7 Тестирование функций и интерфейса доступа 28

4 Исследовательский раздел **32**

4.1 Технические характеристики 32

4.2 Зависимость задержки от числа гонок 32

4.3 Зависимость задержки от конкурентной нагрузки 34

4.4 Обработка результатов и ограничения исследования 35

ЗАКЛЮЧЕНИЕ **37**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **38**

ВВЕДЕНИЕ

Результаты гоночных соревнований образуют систему связанных фактов: позиция на финише относится к определённому заезду, автомобиль выступает за команду, а в его экипаже может находиться несколько гонщиков. Отдельный протокол позволяет определить победителя гонки, однако для сравнения выступлений за сезон требуется объединить результаты разных этапов и вычислить зачётные очки.

При накоплении протоколов в разрозненных документах одни и те же сведения о командах, трассах и гонщиках повторяются. Исправление названия команды или ошибочной позиции приходится согласовывать между документами. База данных позволяет отделить постоянные характеристики участников от результатов выступлений и формировать итоговые показатели по запросу.

Цель работы — разработать базу данных для хранения и обработки результатов гоночных заездов мировых чемпионатов. Для достижения цели поставлены следующие задачи:

1) проанализировать существующие решения, уточнить требования и ограничения предметной области;
2) сравнить модели хранения данных и выбрать подходящую для поставленной задачи;
3) спроектировать базу данных, ограничения целостности и ролевую модель, проверить соответствие 3НФ;
4) спроектировать способы взаимодействия с БД и функции начисления очков и получения статистики;
5) описать реализацию сущностей, функций и полномочий, интерфейс доступа и внешнее кэширование в Redis;
6) определить методы тестирования и проверить корректность расчётов;
7) исследовать зависимость времени выполнения от объёма данных и количества параллельных запросов.

В ходе разработки формализуются связи между соревнованиями, участниками и результатами, проектируются таблицы и реализуются расчётные функции. Функция начисления очков универсальна для чемпионатов, сведения о которых хранятся в БД: она учитывает место, тип и продолжительность гонки. Исследовательская часть содержит методику, результаты измерений и анализ влияния объёма данных и конкурентной нагрузки на время ответа.

1 Аналитический раздел

1.1 Анализ существующих решений

Работа с результатами гоночных чемпионатов включает публикацию протоколов отдельных заездов, получение итогов сезона и сравнение выступлений участников. Для этих задач используются официальные сайты соревнований и программные сервисы спортивных данных. При их сопоставлении существенны охват чемпионатов, представление классов и экипажей, доступность исторических результатов и способ получения данных.

Официальный архив Formula 1 предоставляет выбор сезона и Гран-при, результаты гонок и зачёты гонщиков и команд [1]. Такой способ удобен для просмотра опубликованных итогов: пользователю не требуется самостоятельно собирать протоколы и вычислять классификацию. Данный архив охватывает только Formula 1 и не объединяет результаты других чемпионатов.

На сайте FIA World Endurance Championship (WEC) сезон представлен последовательностью этапов и несколькими зачётами. Публикуются классификации производителей, команд и гонщиков; рядом с очками, полученными на этапе, показана сумма за сезон. Раздельные классификации категорий Hypercar и LMGT3 показывают, что положение участника нужно рассматривать в контексте его класса [2]. Архив позволяет изучать выступления экипажей в WEC, но не объединяет эти результаты с протоколами других чемпионатов.

Официальный раздел IMSA WeatherTech SportsCar Championship позволяет выбирать сезон, этап и сессию, в том числе квалификацию и гонку. В протоколе приведены общая позиция, место в классе, номер автомобиля, гонщики экипажа, команда и число пройденных кругов [3]. Сезонные зачёты опубликованы отдельно для гонщиков, производителей и команд; для гонщиков доступны очки по этапам и итоговая сумма [4]. Эти материалы помогают сопоставлять результаты внутри класса и за весь сезон, но охватывают только соревнования IMSA.

Jolpica F1 предоставляет программный интерфейс получения сведений о трассах, гонщиках, командах, гонках, квалификациях и зачётах. Данные выдаются в структурированном виде; предусмотрены параметры постраничного получения результатов [5]. Это позволяет автоматизировать загрузку сведений о Formula 1. Для совместного анализа с результатами WEC и IMSA потребуется отдельная обработка. Кроме того, приложение, обращающееся к внешнему API, зависит от его доступности, ограничений запросов и принятого формата данных.

Сравнение решений приведено в таблице 1.1.

Таблица 1.1 — Сравнение источников гоночных результатов

Источник	Способ доступа	Охват	Представление результатов
Архив Formula 1	Сайт	Formula 1	Гонки, зачёты гонщиков и команд
Архив FIA WEC	Сайт	WEC	Этапы, классы, сезонные зачёты
Раздел IMSA	Сайт	IMSA	Сессии, позиции в классе, экипажи и зачёты
Jolpica F1	API	Formula 1 (есть возможность распространения на другие серии)	Гонки, квалификации и зачёты в структурированном виде

Примеры WEC и IMSA показывают практическую значимость класса, состава экипажа и раздельного представления результата этапа и сезонного зачёта. Для поставленной задачи требуется хранилище, объединяющее протоколы нескольких чемпионатов, проверяющее допустимость изменений и обеспечивающее единообразный расчёт показателей. Официальные архивы и API могут использоваться как источники сведений, но сами по себе не решают задачу совместной обработки данных.

1.2 Уточнение требований к системе

Система предназначена для накопления результатов гоночных заездов мировых чемпионатов и получения статистики по соревнованиям, командам и гонщикам. Входные сведения включают организатора и год чемпионата, характеристики заездов и трасс, информацию об автомобилях и участниках, составы экипажей, места в квалификациях и на финише.

Функциональные требования определяются следующими пользовательскими действиями:

1) вводить и уточнять сведения о чемпионатах, трассах, заездах, командах, автомобилях и гонщиках;
2) регистрировать участие автомобиля под заданным номером и указывать состав его экипажа;
3) записывать и исправлять протоколы квалификации и финиша;
4) просматривать составы экипажей и результаты выбранного заезда;
5) получать сумму очков участника в чемпионате и последовательность начислений по гонкам;
6) рассчитывать количество гонок, побед, подиумов, очков и поул-позиций, лучшие результаты гонщика и команды;
7) определять положение участников в зачёте своего гоночного класса.

В итоговой классификации должны отображаться команда, стартовый номер, автомобиль, гоночный класс и начисленные очки. Статистика гонщика и команды должна содержать

сопоставимые показатели. При отсутствии выступлений количество гонок и сумма очков равны нулю, а лучшее место не определено. Отсутствие результата необходимо отличать от финиша за пределами очковой зоны.

К требованиям целостности относятся однократный учёт каждого результата, существование соответствующих соревнований и участников, допустимость введённых позиций и одинаковый порядок подсчёта очков для всех хранимых чемпионатов. Исправление протокола должно отражаться в последующих расчётах. Частично введённый протокол не должен восприниматься как успешно завершённая операция загрузки.

Доступ к просмотру, изменению спортивных результатов и сопровождению системы должен разграничиваться. Пользователь, формирующий статистику, не должен иметь возможность случайно изменить результаты. Проверка полномочий должна распространяться и на обращение к данным через программный интерфейс.

Повторное получение неизменившейся статистики допускает использование внешнего кэша в оперативной памяти. Кэшированные сведения должны иметь ограниченный срок актуальности и обновляться после изменения результатов. Основные данные должны оставаться доступными при отказе кэша. Для проверки производительности требуется измерить время расчётов при увеличении числа гонок и числа одновременно выполняемых запросов.

1.3 Ограничения предметной области

Чемпионат рассматривается как соревнование определённого организатора за один год. Сочетание организатора и года достаточно для различения учитываемых чемпионатов.

Единицей выступления является автомобиль, заявленный командой под определённым номером. На нём может выступать один гонщик или экипаж из нескольких человек. Состав экипажа принимается постоянным для всех выступлений этой заявки. При переходе гонщика, смене команды или автомобиля последующие выступления учитываются отдельно, чтобы не менять принадлежность ранее полученных результатов.

Для обработки протоколов принимаются следующие ограничения:

— результат квалификации и результат гонки учитываются раздельно;

— в каждой сессии одна заявка имеет не более одной итоговой позиции;

— позиция представляет положительное целое число и относится к своему классу;

— одинаковые места в разных классах допустимы;

— дистанция задаётся положительным числом кругов либо продолжительностью в целых часах;

— отсутствие результата не приравнивается к нулевому месту;

— стартовый номер сохраняет своё написание, поэтому номера «01» и «1» различаются.

При подсчёте очков учитываются место финиша, тип и продолжительность гонки. Один алгоритм применяется ко всем чемпионатам, сведения о которых хранятся в БД. Начисление выполняется отдельно за каждый финиш с последующим суммированием. Результат экипажа

полностью учитывается в статистике каждого входящего в него гонщика. При этом число гонок команды определяется по различным заездам, а число подиумов — по результатам её автомобилей: в одном заезде команда может занять несколько призовых мест.

1.4 Анализ баз данных по модели хранения данных

1.4.1 Иерархическая и сетевая модели

Иерархическая модель представляет данные в виде дерева. Она удобна для последовательности «чемпионат — этап — результат», но участие одной команды и одного гонщика во многих соревнованиях не укладывается в единственную ветвь. Повторение сведений в разных ветвях усложняет исправление общих характеристик участника.

Сетевая модель допускает несколько направлений перехода между записями. Она позволяет представить участие в разных соревнованиях без обязательного дублирования каждого объекта, однако получение отчёта связано с выбором путей навигации. Для задачи с меняющимися условиями отбора и большим числом итоговых показателей удобнее декларативное описание выборки.

1.4.2 Реляционная модель

Реляционная модель представляет данные в виде отношений. Выборка задаётся через условия отбора, соединение, группировку и агрегирование. Такой подход соответствует расчёту очков за сезон, статистики участника и положения в зачёте. Ограничения целостности позволяют проверять согласованность данных независимо от интерфейса ввода. Цена такого представления — необходимость соединять сведения при построении развёрнутого результата и учитывать стоимость этих операций при росте объёма данных.

1.4.3 Документная и графовая модели

В документной модели протокол одного заезда можно хранить как самостоятельный документ с вложенным списком результатов. Это удобно для чтения протокола целиком. Однако статистика за несколько сезонов требует согласования обозначений участников во многих документах. Повторение описательных данных увеличивает число мест, которые нужно изменить при исправлении сведений о команде или автомобиле.

Графовая модель представляет объекты вершинами, а отношения между ними — рёбрами. Она полезна для исследования цепочек переходов гонщиков между командами и совместных выступлений. В поставленной задаче преобладают числовые агрегаты и ранжирование, поэтому выразительность обходов графа не является определяющим преимуществом.

1.4.4 Объектная, многомерная модели и «ключ — значение»

Объектная модель сохраняет структуру и идентичность объектов программы. Для рассматриваемой задачи это не даёт отдельного преимущества: основной объём обработки связан с вычислением сводных показателей, а не с сохранением поведения объектов.

Многомерная модель организует показатели по измерениям: сезону, участнику, классу. Она удобна для аналитических срезов. Поскольку исправление протокола требует обновления агрегатов, такое представление целесообразно как дополнение к подробным результатам.

Модель «ключ — значение» предоставляет доступ к готовому результату по уникальному ключу и подходит для кэширования статистики. Получение произвольных срезов и пересчёт зачётов требуют дополнительной прикладной логики. Поэтому модели основного хранилища и кэша следует выбирать раздельно.

1.4.5 Выбор модели

Основные различия с точки зрения поставленной задачи приведены в таблице 1.2.

Таблица 1.2 — Применимость моделей хранения к обработке гоночных результатов

Модель	Подходящая операция	Ограничение для задачи
Иерархическая	Чтение результатов одного чемпионата	Повторное участие объектов в разных ветвях
Сетевая	Переходы между связанными записями	Зависимость обработки от путей навигации
Реляционная	Отбор, суммирование и ранжирование	Затраты на соединения больших наборов
Документная	Чтение полного протокола	Согласование общих сведений в документах
Графовая	Анализ цепочек участия	Основная задача требует числовых агрегатов
Объектная	Сохранение объектов программы	Привязка представления к объектной структуре
Многомерная	Аналитические срезы	Обновление агрегатов при исправлении фактов
Ключ — значение	Чтение готового результата	Нет самостоятельного расчёта произвольного зачёта

Вывод

Анализ существующих решений позволил уточнить требования и границы предметной области. Для основной базы выбрана реляционная модель, объединяющая проверку целостности и расчёт агрегатов. Для внешнего кэша выбрана модель «ключ — значение», обеспечивающая доступ к готовой статистике.

2 Конструкторский раздел

2.1 Проектирование базы данных

В базе выделены 13 отношений. Шесть справочных сущностей описывают организаторов, трассы, производителей, гонщиков, команды и классы. Чемпионат, гонка, автомобиль и заявка представляют основные объекты соревнования. Состав экипажа, финиш и квалификация оформляются отдельными отношениями.

Связи «чемпионат — гонка», «трасса — гонка», «команда — заявка» и «автомобиль — заявка» имеют кратность один ко многим. Гонщики и заявки, а также заявки и заезды участвуют в связях многие ко многим. Разработанная концептуальная схема приведена на рисунке 2.1. На ней результат связан с машиной-участником, а принадлежность гонщика к экипажу представлена самостоятельной связью. При переходе к реляционной модели эти связи представлены отношениями состава экипажа, финиша и квалификации.

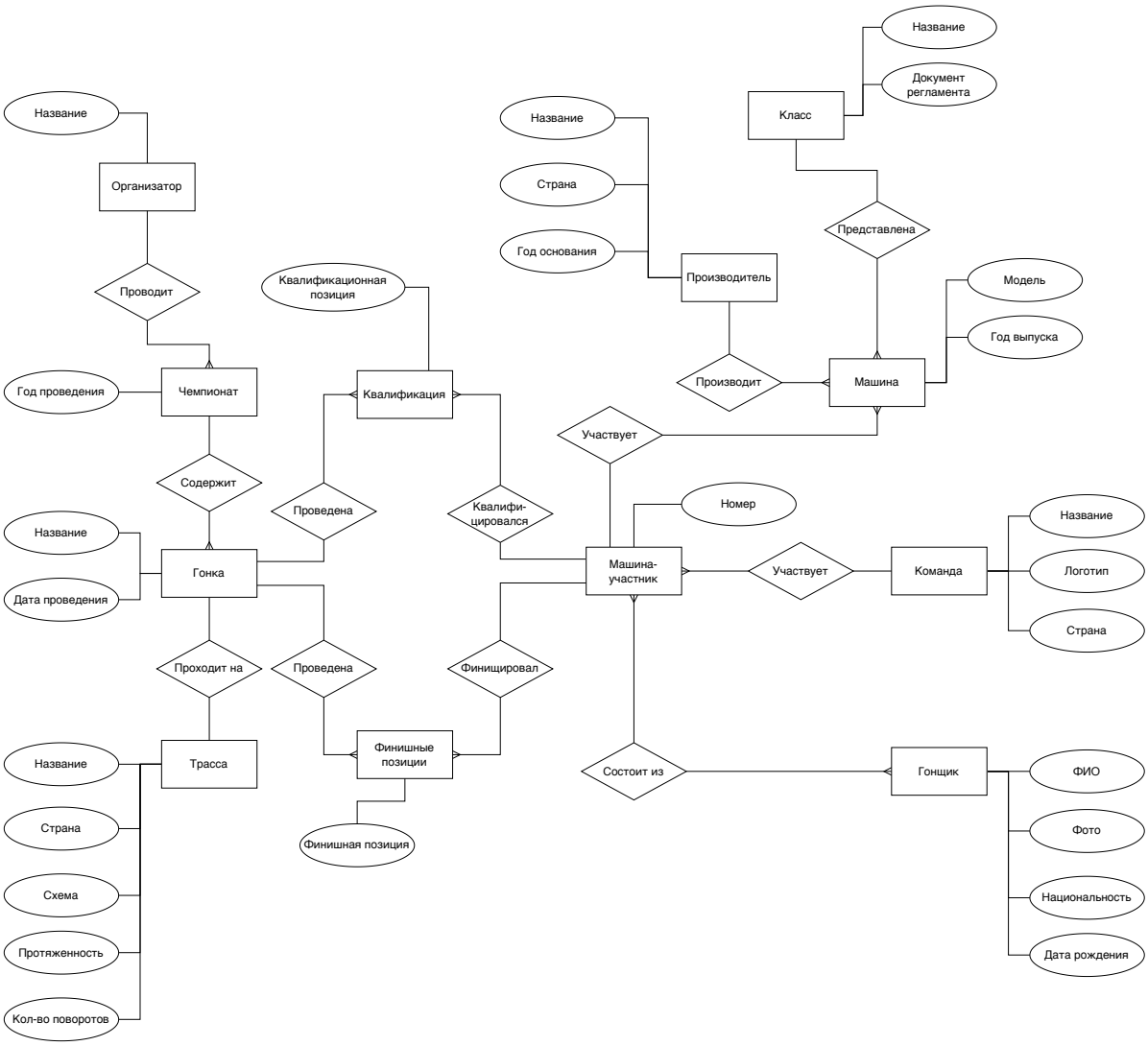


Рисунок 2.1 — Концептуальная схема предметной области

2.1.1 Справочные сущности

Атрибуты справочных сущностей перечислены в таблице 2.1. Значение идентификатора не имеет спортивного смысла.

Таблица 2.1 — Справочные сущности и их атрибуты

Сущность	Атрибуты помимо первичного ключа `id`
Организатор	`name`: название организатора или серии
Трасса	`name`, `country`: название и страна; `schema`: изображение; `lap_length`: длина круга, м; `turns`: число поворотов
Производитель	`name`, `country`; `year_of_foundation`: год основания
Гонщик	`name`, `nationality`; `birthday`: дата рождения; `photo`: фотография
Команда	`name`, `country`; `logo`: логотип
Класс	`name`: название класса; `docs`: текст либо ссылка на описание регламента

2.1.2 Чемпионаты, заезды и участники

В таблице 2.2 приведены сущности, связывающие справочные сущности. Ссылочные столбцы обязательны. Продолжительность задаётся количеством кругов или часов, в зависимости от типа гонки (спринт или гонка на выносливость).

Таблица 2.2 — Основные сущности соревнований

Сущность	Атрибуты помимо `id`
Чемпионат	Ссылка на организатора; `year`: год сезона
Гонка	Ссылки на чемпионат и трассу; `name`, `date`; `type`: способ задания дистанции; `duration`: число кругов либо часов
Автомобиль	Ссылки на производителя и класс; `model`: модель; `year_of_production`: год выпуска
Заявка	Ссылки на автомобиль и команду; `number`: стартовый номер

Стартовые номера «01» и «1» считаются различными. Номер не является первичным ключом заявки: одинаковые номера могут встретиться в разных классах и сезонах. Однако получение результата по номеру однозначно в рамках одного чемпионата.

Сущности связей представлены в таблице 2.3. Для результата естественным ключом является пара «заезд, заявка». Наличие только внешних ключей недостаточно: без уникальности пары можно дважды внести один финиш и удвоить начисленные очки.

Таблица 2.3 — Сущности связей и результатов

Отношение	Первичный ключ	Прочие атрибуты
Состав экипажа	(заявка, гонщик)	Отсутствуют
Финиш	(гонка, заявка)	`pos`: место на финише
Квалификация	(гонка, заявка)	`pos`: место в квалификации

2.2 Ограничения целостности

Ограничения разделяются на ключевые, ссылочные и доменные. Первичный ключ определяет запись; внешний ключ запрещает ссылки на отсутствующие объекты; доменная проверка ограничивает допустимые значения. В таблице 2.4 указаны ограничения разработанной базы данных.

Таблица 2.4 — Реализуемые проверки целостности

Объект	Правило
Все основные сущности	Уникальный и непустой `id`
Состав экипажа и результаты	Уникальные составные ключи; обязательные внешние ключи
Финиш и квалификация	`pos > 0`
Гонка	`type IN (1,2)`, `duration > 0`
Трасса	`lap_length > 0`, `turns >= 0`
Чемпионат	Год от 1900 до 2100
Производитель и автомобиль	Положительный год основания и выпуска
Связанные записи	Ссылочная целостность и `ON DELETE CASCADE`

Для результатов выбраны составные первичные ключи, запрещающие повтор пары «гонка, заявка». Аналогично ключ состава экипажа исключает повторное включение одного гонщика в экипаж. Доменные проверки задаются средствами `CHECK` и выполняются при добавлении и изменении записей.

Для внешних ключей выбрано каскадное удаление. При удалении гонки удаляются её квалификации и финиши; при удалении команды удаляются её заявки и связанные результаты. Это поддерживает ссылочную целостность, но позволяет потерять историю соревнований.

2.3 Проверка соответствия третьей нормальной форме

Проверка нормализации проводится по функциональным зависимостям атрибутов. Отношение находится в третьей нормальной форме, если для каждой нетривиальной зависимости $X \to A$ детерминант X является суперключом либо атрибут A входит в некоторый потенциальный ключ. Проверка опирается на принятые предметные ограничения; само наличие столбца идентификатора недостаточно для доказательства 3НФ.

Первая нормальная форма выполняется: каждый атрибут содержит одно значение своего домена. Состав экипажа не записывается списком в одной ячейке, а представлен отдельными записями. Результаты разных гонок также хранятся раздельно.

Для шести справочных сущностей и четырёх основных сущностей все описательные атрибуты зависят от первичного ключа `id`. Частичная зависимость от части такого ключа невозможна. Например,

$$\text{ID автомобиля} \;\to\; (\text{производитель}, \text{модель}, \text{год выпуска}, \text{класс}). \tag{2.1}$$

Название производителя определяется его собственным ключом и не повторяется у автомобилей. Аналогично название команды не хранится в каждом результате.

Для финиша основная зависимость имеет вид

$$(\text{гонка}, \text{заявка}) \;\rightarrow\; \text{позиция}. \tag{2.2}$$

Отдельная гонка не определяет позицию без участника, а участник не определяет позицию без гонки. Следовательно, зависимость относится ко всему составному ключу. Для квалификаций справедливо аналогичное рассуждение.

В отношении состава экипажа оба атрибута входят в составной ключ, неключевых атрибутов нет. Следовательно, частичные зависимости отсутствуют во всех 13 отношениях: выполнены условия второй нормальной формы.

Транзитивные зависимости устранены разделением описательных сведений. Название организатора хранится только у организатора, параметры трассы — у трассы, характеристики автомобиля — у автомобиля. Гонка связана с чемпионатом и трассой, но не повторяет их описания. Заявка не дублирует названия команды, производителя и класса. В результатах хранится позиция, а очки и статистика вычисляются по запросу.

При принятом наборе зависимостей каждый неключевой атрибут определяется ключом своего отношения и не зависит от другого неключевого атрибута. Таким образом, разработанная структура соответствует 3НФ. Это свойство устраняет избыточное повторение описательных данных, но не заменяет проверки спортивных правил и согласованности состава экипажа.

2.4 Проектирование ролевой модели базы данных

Полномочия разделяются по характеру работы с данными. Сопровождение структуры, ввод результатов и получение статистики требуют разных наборов прав. Для базы проектируются три роли: владелец, оператор и аналитик. Каждая роль определяет доступные действия с данными и структурой.

Владелец сопровождает таблицы, ограничения, индексы и функции, а также ведёт записи справочных сущностей и календарь заездов. Оператор формирует заявки и экипажи, вносит и исправляет результаты. Аналитик получает протоколы и статистику. Состав полномочий приведён в таблице 2.5.

Таблица 2.5 — Проектируемые полномочия ролей

Роль	Доступные операции	Границы полномочий
Владелец	Владение объектами БД, изменение структуры, записей справочных сущностей и календаря	Сопровождение в пределах базы
Оператор	Чтение; вызов функций; добавление и изменение заявок и экипажей; изменение протоколов	Нет права менять структуру, удалять записи справочных сущностей и заявки
Аналитик	Чтение данных и выполнение расчётных функций	Нет прав изменения данных и структуры

Оператор может добавлять и изменять заявки и состав экипажа, а также добавлять, изменять и удалять результаты финиша и квалификации. Право удаления результата позволяет исправить ошибочно внесённый финиш, не удаляя саму гонку.

Оператор и аналитик могут вызывать расчётные функции и читать необходимые для них данные. Функции не должны расширять полномочия вызвавшего пользователя. Создание объектов и вызов функций ограничиваются заданной матрицей полномочий.

Удаление записей справочных сущностей сохраняется за владельцем: каскадные внешние ключи могут привести к удалению связанных результатов. Назначенные права не заменяют проверку смысла изменения. Например, оператор не должен исправлять состав старой заявки, если изменение относится только к последующим выступлениям.

2.5 Проектирование способов взаимодействия с базой данных

Интерфейс доступа включает ввод протоколов, просмотр подробных результатов и запрос статистики. Для каждого действия определяется набор параметров и возвращаемых данных. Вычислительные правила размещаются в функциях БД, поэтому разные клиентские интерфейсы получают одинаковые суммы и показатели.

При запросе статистики пользователь выбирает чемпионат, гонщика либо команду. Прикладной уровень проверяет формат параметров, передаёт их в SQL-запрос и отображает полученный результат. Идентификатор используется для однозначного выбора записи; наименования и стартовые номера служат для представления пользователю. Строковые параметры передаются отдельно от текста SQL.

Загрузка протокола рассматривается как одна операция. Все позиции записываются в транзакции, которая фиксируется только после успешного выполнения команд. При нарушении целостности выполняется откат. Наличие двух машин одной команды или нескольких гонщиков в экипаже учитывается функциями, а не повторным сложением показателей в клиенте.

Для повторных запросов предусматривается внешний кэш. В нём находятся только результаты чтения, которые можно восстановить из основной базы. Изменение протоколов всегда направляется в базу данных. После успешной фиксации прикладной уровень должен исключить использование устаревших кэшированных показателей. Компонентная схема приведена на рисунке 2.2.



Рисунок 2.2 — Проектируемое взаимодействие компонентов

При недоступности кэша должен выполняться запрос чтения из базы данных. Ошибки основной базы нельзя заменять ранее сохранённым успешным ответом: пользователю необходимо сообщить о невозможности выполнить операцию. Запрос, которому требуется актуальное состояние непосредственно после исправления протокола, выполняется без кэша.

2.6 Проектирование функций расчёта очков и статистики

2.6.1 Начисление очков и классификация

Обозначим место заявки через p. Базовая шкала $b(p)$ задаётся таблицей 2.6. Для мест за пределами первой десятки начисляется ноль очков.

Таблица 2.6 — Базовая шкала расчётной функции

Место	1	2	3	4	5	6	7	8	9	10
Очки	25	18	15	12	10	8	6	4	2	1

Для типа гонки τ и её продолжительности d коэффициент определяется как

$$k(\tau, d) = \begin{cases} 1{,}5, & \tau = 2, \quad 6 < d < 24, \\ 2, & \tau = 2, \quad d \geq 24, \\ 1, & \text{в остальных случаях.} \end{cases} \tag{2.3}$$

Очки за отдельный результат вычисляются по формуле

$$s(p, \tau, d) = \lceil b(p)\, k(\tau, d) \rceil\,. \tag{2.4}$$

Округление вверх выполняется для каждой гонки до суммирования. Например, третье место в восьмичасовом заезде даёт $\lceil 15 \cdot 1{,}5 \rceil = 23$ очка. Два таких результата дают 46, тогда как округление суммы произведений дало бы 45. Эта разница включена в методику тестирования.

Формула (2.4) позволяет унифицировать начисление очков для чемпионатов, которые хранятся в БД. Функция универсальна в пределах этих чемпионатов: коэффициенты и порядок округления одинаково применяются ко всем результатам.

Пусть $F_{c,a}$ — множество результатов заявки a в чемпионате c. Тогда сумма очков заявки равна

$$P_{c,a} = \sum_{f \in F_{c,a}} s(p_f, \tau_f, d_f). \tag{2.5}$$

В результат функции входят только заявки, имеющие хотя бы один финиш в выбранном чемпионате. Заявка с результатом вне очковой зоны присутствует с нулевой суммой; заявка без финишей отсутствует. Сортировка выдачи производится по убыванию очков.

Схема вычисления приведена на рисунке 2.3. В алгоритме сначала ограничивается набор гонок, затем рассчитывается вклад каждого результата и только после этого формируются суммы.

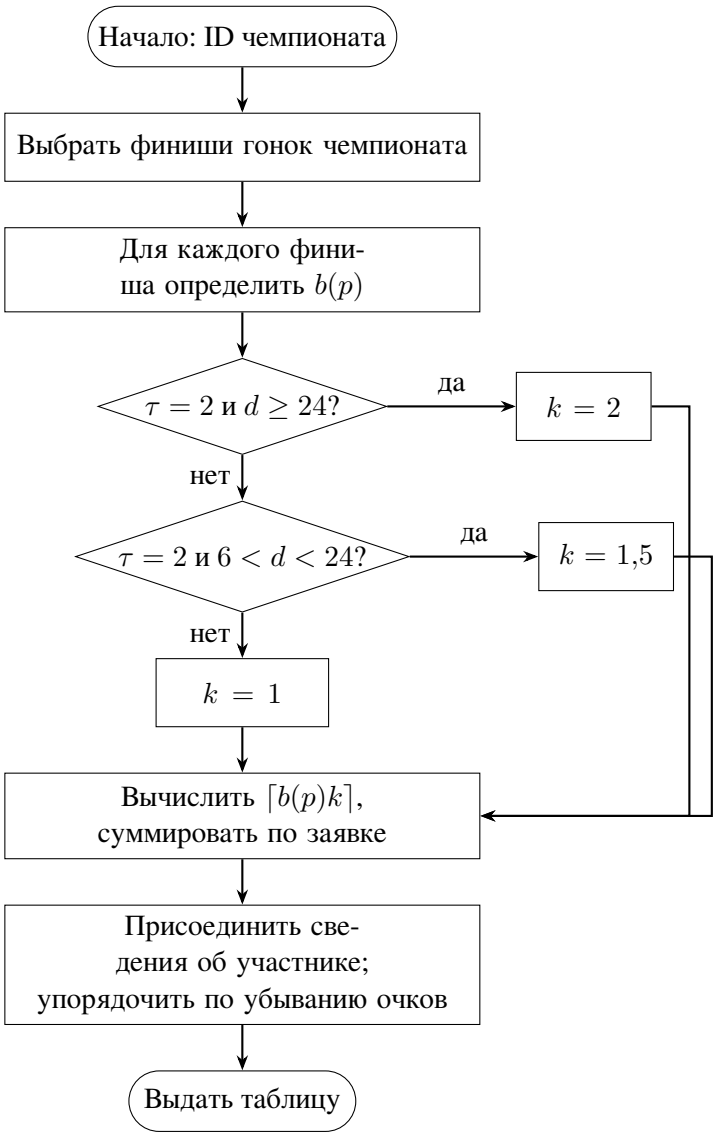


Рисунок 2.3 — Алгоритм расчёта очков заявок

2.6.2 Статистика гонщиков и команд

Для гонщика сначала выбираются все заявки, в которых он состоит. По ним формируются два независимых набора: финиши и квалификации. Прямое соединение этих наборов до агрегации могло бы размножить строки, поэтому показатели вычисляются отдельными подзапросами.

Для команды выбираются все её заявки. Если две машины команды финишировали в одной гонке, число гонок команды увеличивается на один, а число финишей — на два. Счётчики побед и подиумов подсчитывают строки результатов: команда может получить несколько подиумов за один заезд.

Семантика выходных полей приведена в таблице 2.7. Минимум по пустому набору остаётся неопределённым. Замена его нулём означала бы несуществующее «нулевое место» и потому не выполняется.

Таблица 2.7 — Показатели статистических функций

Поле	Смысл
`total_races`	Число различных гонок с финишем
`total_wins`	Число финишей на первом месте
`total_podiums`	Число финишей с позицией не хуже третьей
`total_points`	Сумма очков всех выбранных финишей
`total_poles`	Число первых мест в квалификациях
`best_finish`	Минимальная позиция на финише
`best_qualifying`	Минимальная позиция квалификации
`championship_wins`	Число пар «чемпионат, класс», в которых получена первая позиция расчётного зачёта
`best_championship_position`	Лучшая позиция среди соответствующих зачётов

Поле `championship_wins` содержит число первых мест в расчётных зачётах. Поскольку в базе нет признака завершения чемпионата, этот показатель отражает лидерство по имеющимся данным и не обязательно совпадает с количеством официально завоёванных титулов. Для окончательного зачёта нужны утверждённые полные результаты.

Общая схема получения статистики приведена на рисунке 2.4. Вычисление чемпионатных позиций требует сравнения с соперниками, поэтому объём прочитанных данных может превышать число финишей выбранного гонщика.

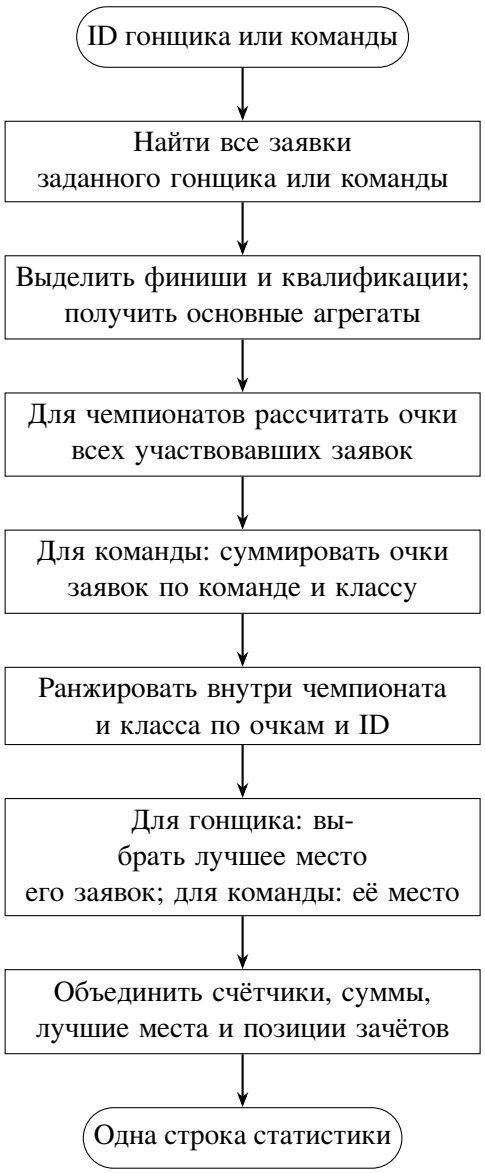


Рисунок 2.4 — Получение статистики гонщика и команды

Функция детализации очков устроена проще: отбираются финиши заданного чемпионата, проверяется соответствие номера условию `LIKE`, вычисляются очки по формуле (2.4), строки сортируются по дате гонки. Схема этой последовательности приведена на рисунке 2.5.



Рисунок 2.5 — Получение детализации начисленных очков

2.7 Индексы и согласованность операций

Первичные ключи создают индексы для поиска по идентификатору и уникальности составных пар. Дополнительно предусмотрены индексы для гонок по чемпионату, для финишей и квалификаций по заявке и гонке, для состава экипажа по гонщику и заявке, для заявок по команде. Они соответствуют начальным условиям выборок расчётных функций.

Индексы не гарантируют ускорение любой выборки. При чтении существенной части таблицы последовательное сканирование может оказаться дешевле.

Загрузка протокола должна выполняться транзакционно: все позиции одного заезда записываются в одной транзакции. Это исключает чтение частично загруженного протокола после промежуточных фиксаций. Для нескольких последовательных отчётных запросов, которым требуется один снимок данных, может использоваться уровень `REPEATABLE READ`; при `READ COMMITTED` последовательные команды могут видеть разные зафиксированные состояния. Сценарий измерений использует неизменяемый набор данных, поэтому конкурентные обновления не входят в его условия.

Вывод

Спроектированы структура базы данных, ограничения целостности и индексы; проверено соответствие отношений 3НФ. Определены роли владельца, оператора и аналитика, способы взаимодействия с БД и место внешнего кэша. Представлены алгоритмы начисления очков, детализации результатов и получения статистики гонщиков и команд.

3 Технологический раздел

3.1 Средства реализации

Для основной базы данных выбрана PostgreSQL. Первичные и внешние ключи, проверки допустимых значений и транзакции обеспечивают согласованность результатов. Соединения, общие табличные выражения и оконные функции используются для получения статистики и мест в зачёте. SQL применяется для описания сущностей и запросов, PL/pgSQL — для табличных расчётных функций [6].

Средством непосредственного обращения к БД служит SQL-клиент `psql`. Он позволяет вводить данные, выполнять запросы и вызывать функции с заданными параметрами. Для прикладного интерфейса определяются те же операции: просмотр протокола, подсчёт очков, статистика гонщика и команды. Внешнее кэширование рассматривается на базе InMemory СУБД Redis, хранящей результаты повторяющихся запросов.

Проверки функций и ограничений выполнены на PostgreSQL 14.17. Нагрузочные исследования проводились средствами `pgbench`. Python 3 используется для формирования наборов данных для замеров и обработки протоколов измерений. Результаты исследования относятся к выполнению SQL-функций в PostgreSQL.

3.2 Реализация сущностей базы данных

Реляционная структура включает 13 таблиц. Их назначение и способ представления данных приведены в таблице 3.1. Отдельное хранение справочных сущностей позволяет исправлять сведения об участнике без изменения всех его результатов.

Таблица 3.1 — Группы сущностей базы данных

Группа	Таблицы	Назначение
Справочные сущности	`organizer`, `track`, `manufacturer`, `driver`, `team`, `raceclass`	Постоянные характеристики соревнований и участников
Соревнования	`championship`, `race`	Сезон, трасса, дата и параметры заезда
Автомобили и заявки	`car`, `car_p`	Модель машины и её участие за команду под номером
Состав экипажа	`team_p`	Включение гонщика в заявку
Результаты	`finish`, `qualifying`	Позиции заявки на финише и в квалификации

Идентификаторы справочных сущностей и основных объектов имеют тип `integer`, для автоматической нумерации используется `SERIAL`. Даты гонок и рождения гонщиков представлены типом `date`. Наименования, страны и стартовые номера имеют тип `text`. Фотографии, логотипы и схемы трасс сохраняются в полях `bytea`.

Таблицы результатов содержат ссылки на заезд и заявку, а также позицию. Первичный ключ из двух ссылок исключает повторное начисление за один и тот же финиш. Поле pos обязательно и допускает только положительные значения. Пример определения таблицы приведён в листинге 3.1.

Листинг 3.1 — Описание сущности результата гонки

```
CREATE TABLE finish (
    race integer NOT NULL REFERENCES race(id)
        ON DELETE CASCADE,
    car_p integer NOT NULL REFERENCES car_p(id)
        ON DELETE CASCADE,
    pos integer NOT NULL CHECK (pos > 0),
    PRIMARY KEY (race, car_p)
);
```

Для qualifying используются аналогичные столбцы и ограничения. В team_p ключ образуют car_p и driver. Для типа гонки задано условие type IN (1,2), для продолжительности — duration > 0. Проверки числовых характеристик трассы и годов не допускают значений вне выбранных доменов.

Индексы по чемпионату гонки, заявке результата, участию гонщика и команде заявки соответствуют условиям отбора статистических функций. Суммарные очки не сохраняются в отдельном столбце: они вычисляются по позициям и характеристикам гонок. Это исключает необходимость вручную согласовывать сохранённый итог с исправленным протоколом.

3.3 Реализация функций базы данных

3.3.1 Очки чемпионата и детализация

Функция calculatepersonalpoints(integer) соединяет финиши с гонками, ограничивает выборку чемпионатом и группирует данные по car_p.id. Полученная сумма дополняется сведениями из записей справочных сущностей. Внешняя выборка возвращает семь столбцов. Сумма приводится к integer; поэтому очень большие искусственные наборы не должны приводить к превышению диапазона этого типа.

Основная часть выражения начисления соответствует листингу 3.2. Использование точных числовых литералов исключает необходимость округлять двоичное представление числа $1{,}5$ на стороне клиента.

Функция caclulatepersonalpointstable возвращает начисления по отдельным гонкам. Первый аргумент задаёт чемпионат, второй — шаблон стартового номера. Значение '01' отбирает строки с этим номером, а '0%' использует маску.

Листинг 3.2 — Расчёт очков одного результата

```
CEIL(
    CASE f.pos
        WHEN 1 THEN 25 WHEN 2 THEN 18
        WHEN 3 THEN 15 WHEN 4 THEN 12
        WHEN 5 THEN 10 WHEN 6 THEN 8
        WHEN 7 THEN 6  WHEN 8 THEN 4
        WHEN 9 THEN 2  WHEN 10 THEN 1
        ELSE 0
    END *
    CASE
        WHEN r.type = 2 AND r.duration > 6
             AND r.duration < 24 THEN 1.5
        WHEN r.type = 2 AND r.duration >= 24 THEN 2
        ELSE 1
    END
)::int
```

3.3.2 Агрегирование статистики

Функции гонщика и команды строят несколько общих табличных выражений. Сначала формируется множество заявок, затем — результаты и квалификации. Вычисление чемпионатных позиций использует `CROSS JOIN LATERAL`: для каждого чемпионата вызывается функция очков. После ранжирования выбираются только позиции интересующего участника.

Ключевые агрегаты приведены в листинге 3.3. `COALESCE` используется только там, где пустая сумма по смыслу равна нулю; значения лучших мест остаются неопределёнными.

Листинг 3.3 — Принцип вычисления основных показателей

```
SELECT
    (SELECT COUNT(DISTINCT race_id) FROM race_results),
    (SELECT COUNT(*) FROM race_results
        WHERE finish_pos = 1),
    (SELECT COUNT(*) FROM race_results
        WHERE finish_pos <= 3),
    (SELECT COALESCE(SUM(points), 0) FROM race_results),
    (SELECT COUNT(*) FROM qualifying_results WHERE pos = 1),
    (SELECT MIN(finish_pos) FROM race_results),
    (SELECT MIN(pos) FROM qualifying_results);
```

3.4 Роли и привилегии

Ролевая модель выражается через группы полномочий PostgreSQL. Роль `racing_owner` предназначена для владения объектами, `racing_operator` — для работы с заявками и протоколами, `racing_analyst` — для чтения и расчётов. Оператору необходимы права аналитика, дополненные разрешениями изменения рабочих данных.

SQL-описание доступа использует `GRANT` и `REVOKE`. Права на таблицы, последовательности и функции задаются отдельно [6]. Фрагмент, соответствующий проектируемой матрице,

приведён в листинге 3.4. В нём предполагается, что рабочая схема public выделена для данной базы.

Листинг 3.4 — SQL-описание полномочий оператора и аналитика

```
CREATE ROLE racing_analyst NOLOGIN;
CREATE ROLE racing_operator NOLOGIN;
GRANT racing_analyst TO racing_operator;
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO racing_analyst;
GRANT SELECT ON ALL TABLES IN SCHEMA public
    TO racing_analyst;
REVOKE EXECUTE ON ALL FUNCTIONS IN SCHEMA public
    FROM PUBLIC;
GRANT EXECUTE ON FUNCTION
    calculatepersonalpoints(integer),
    caclulatepersonalpointstable(integer, text),
    calculatedriverstats(integer),
    calculateteamstats(integer)
    TO racing_analyst;
GRANT INSERT, UPDATE ON car_p, team_p
    TO racing_operator;
GRANT INSERT, UPDATE, DELETE ON finish, qualifying
    TO racing_operator;
GRANT USAGE ON SEQUENCE car_p_id_seq
    TO racing_operator;
```

Групповые роли не содержат паролей и самостоятельно не используются для входа. Учётная запись получает требуемую группу полномочий. Запрос аналитика на изменение результата должен завершаться отказом, даже если он передан напрямую через SQL-клиент. Функции работают с правами вызывающего пользователя; общие данные для расчётов доступны обеим рабочим ролям.

3.5 Приложение доступа к базе данных

Для приложения доступа определены операции, соответствующие SQL-интерфейсу базы. Прикладной уровень отвечает за выбор параметров, выполнение запроса и представление ответа. Правила начисления очков и агрегирования остаются внутри PostgreSQL, поэтому приложение не должно повторно вычислять их по отображаемым строкам.

Состав операций приведён в таблице 3.2. Просмотр экипажей и протокола выполняется соединением таблиц, остальные операции обращаются к хранимым функциям.

Таблица 3.2 — Операции интерфейса доступа к базе данных

Операция	Параметры	Обращение к БД
Состав экипажей	Отбор по заявке при необходимости	Соединение `team_p`, `car_p`, `driver` с данными об автомобиле и его производителе
Протокол гонки	Гонка	Соединение `finish` с гонкой, заявкой и справочными сущностями
Очки чемпионата	Чемпионат	`calculatepersonalpoints`
Начисления по гонкам	Чемпионат, шаблон номера	`caclulatepersonalpointstable`
Статистика гонщика	Гонщик	`calculatedriverstats`
Статистика команды	Команда	`calculateteamstats`
Ввод результата	Гонка, заявка, позиция	`INSERT` или `UPDATE` таблицы результатов

Работа с отчётом состоит из выбора вида статистики и участника, передачи идентификатора и отображения полученных показателей. Для детального протокола дополнительно выводятся название гонки, дата, команда, автомобиль, класс и позиция. Для лучших мест значение `NULL` отображается как отсутствие результата, а не как ноль.

На уровне SQL пользовательские сценарии выражаются запросами из листинга 3.5. Идентификатор 1 обозначает запись тестового набора; при обращении к рабочим данным используется выбранный пользователем объект.

Листинг 3.5 — Обращение к расчётным функциям

```
SELECT * FROM calculatepersonalpoints(1);
SELECT * FROM caclulatepersonalpointstable(1, '01');
SELECT * FROM calculatedriverstats(1);
SELECT * FROM calculateteamstats(1);
```

При изменении результата приложение должно проверять формат входных значений и обрабатывать ответ PostgreSQL. Нарушение уникальности означает повторную запись результата; нарушение внешнего ключа — ссылку на отсутствующий объект; нарушение доменной проверки — недопустимую позицию или характеристику заезда. Все команды одного протокола объединяются в транзакцию. Сообщение об успехе формируется после её фиксации.

Результат статистической функции для неизвестного идентификатора совпадает с результатом для участника без выступлений: счётчики равны нулю, лучшие позиции не определены. Поэтому проверка существования выбранного гонщика или команды относится к прикладному сценарию и должна предшествовать отображению статистики.

3.6 Внешнее кэширование с использованием Redis

Для внешнего кэширования определена схема *cache-aside*. При чтении прикладной уровень сначала обращается в Redis. Если значения нет, выполняется соответствующая функция

PostgreSQL, а её результат помещается в кэш. Значение Redis является копией результата запроса и может быть удалено без потери спортивных данных [7].

Кэшированию подлежат очки чемпионата и статистика отдельных гонщиков и команд. Для этих запросов параметры однозначно определяются идентификатором объекта. Формат значения — JSON: массив строк классификации либо объект с девятью статистическими показателями. Время жизни принимается равным 60 с. Это параметр проекта кэша, а не величина, полученная при замерах.

Ключ включает название системы, поколение статистики, вид запроса и идентификатор. В таблице 3.3 символ g обозначает текущее поколение, i — идентификатор выбранного объекта.

Таблица 3.3 — Организация ключей внешнего кэша

Шаблон ключа	Содержимое
`racing:stats:epoch`	Номер текущего поколения; без срока жизни
`racing:stats:g:points:i`	Классификация заявок чемпионата
`racing:stats:g:driver:i`	Статистика гонщика
`racing:stats:g:team:i`	Статистика команды

Чтение выполняется командой `GET`, сохранение с ограниченным сроком жизни — командой `SET` с параметром `EX 60` [8]. После успешного изменения данных прикладной уровень увеличивает поколение статистики командой `INCR`. Новые запросы используют другие ключи; значения прежнего поколения удаляются по истечении срока жизни. Изменение одной позиции может повлиять на места соперников, поэтому обновление поколения распространяется на все статистические запросы, а не только на изменённого участника.

Последовательность чтения приведена на рисунке 3.1. Проверка прав и параметров выполняется до обращения к кэшу.



Рисунок 3.1 — Алгоритм обращения к внешнему кэшу

Запись результата и обновление кэша не образуют общей транзакции PostgreSQL и Redis. Между фиксацией изменения и сменой поколения возможно чтение прежнего значения; неуспешное уведомление кэша также оставляет его до истечения срока жизни. Поэтому актуальный протокол после редактирования необходимо запрашивать непосредственно в PostgreSQL. Отказ Redis должен приводить к выполнению обычного SQL-запроса, а не к отказу получения статистики.

Проверка согласованности кэша должна охватывать изменение финиша, квалификации, характеристик гонки, состава экипажа и принадлежности заявки. Во всех этих случаях могут измениться вычисляемые показатели. Эффект внешнего кэширования нельзя выводить из измерений времени SQL-функций: для него требуется отдельно измерять полный путь прикладного запроса.

3.7 Тестирование функций и интерфейса доступа

3.7.1 Методы и классы тестовых данных

Применяются функциональное, интеграционное и негативное тестирование. Функциональные проверки сопоставляют вычисленные очки с заранее заданной таблицей ожидаемых значений. Интеграционные проверки контролируют соединения, различие количества гонок и количества результатов, разделение по классам и передачу результата членам экипажа. Негативные проверки пытаются внести данные, нарушающие ограничения, и проверяют код ошибки SQLSTATE.

Тесты выполняются в отдельной базе с созданными таблицами и функциями, без рабочих данных, в одной транзакции. Сценарий создаёт небольшой набор записей справочных сущностей, результатов и временные функции проверки. Любое несоответствие вызывает исключение и завершает `psql` с ошибкой благодаря `ON_ERROR_STOP`. В конце выполняется `ROLLBACK`; поэтому успешный прогон не оставляет тестовые записи.

Для начисления очков выделены классы эквивалентности: места от первого до десятого с разными ненулевыми значениями, места вне десятки, обычная гонка и гонка на время. Граничные значения длительности: 6, 7, 23, 24 и 25. Они проверяют обе стороны порогов с учётом целочисленного домена. Значение 11 представляет первую позицию без очков. Для типа 1 все эти длительности должны сохранять единичный коэффициент.

Табличный эталон задаётся независимо от выражения `CASE`: для каждого коэффициента явно перечислены ожидаемые начисления по позициям. Результаты всех расчётных функций сравниваются с этим эталоном. Отдельно проверяются соединения таблиц, разделение участников по классам и реакции базы на недопустимые данные.

Основные сценарии приведены в таблице 3.4.

Таблица 3.4 — Проверяемые классы ситуаций

Сценарий	Ожидаемое поведение
Места 1–11, длительности 6/7/23/24/25	Совпадение с эталоном во всех функциях
Чемпионат без результатов	Пустая таблица очков
Гонщик без выступлений	Нулевые счётчики; лучшие места `NULL`
Две машины одной команды	Число гонок не удваивается, очки суммируются
Два гонщика в одном экипаже	Одинаковые результаты и начисления
Участники разных классов	Независимые позиции в зачётах
Равные суммы очков	Порядок по идентификатору
Маска стартового номера	Выбор всех соответствующих заявок
Повтор результата или члена экипажа	SQLSTATE `23505`
Несуществующий внешний ключ	SQLSTATE `23503`
Недопустимое числовое значение	SQLSTATE `23514`
`NULL` в позиции	SQLSTATE `23502`
Удаление гонки	Удаление зависимых финишей

3.7.2 Контрольный пример и результаты проверок

Интеграционный набор содержит три гонки длительностью 6, 8 и 24 часа, четыре заявки, три участвующие команды и два класса. Первая и вторая заявки принадлежат одной команде. Четвёртая заявка относится к другому классу и имеет двух гонщиков в экипаже. Позиции

и суммы приведены в таблице 3.5; значения являются заданными тестовыми данными, а не результатами реальных соревнований.

Таблица 3.5 — Контрольные финиши и ожидаемые очки

Заявка	Класс	Место, 6 ч	Место, 8 ч	Место, 24 ч	Очки
1	A	1	3	10	$25 + 23 + 2 = 50$
2	A	2	1	11	$18 + 38 + 0 = 56$
3	A	3	2	1	$15 + 27 + 50 = 92$
4	B	1	1	2	$25 + 38 + 36 = 99$

У первой команды получаются 3 различные гонки, 2 победы, 4 подиума и 106 очков. В квалификационном наборе она имеет 2 первых места. Команда опережает соперника класса A с 92 очками, хотя ни одна из её заявок по отдельности не занимает первое место личной классификации заявок.

Гонщик первой заявки получает 3 гонки, 1 победу, 2 подиума, 50 очков и 1 поул-позицию. Его лучшие финиш и квалификация равны 1, число первых мест в зачётах равно 0, лучшее место в зачёте — 3. Оба гонщика четвёртой заявки получают по 99 очков: результат экипажа учитывается в статистике каждого его члена.

На PostgreSQL 14.17 выполнены автоматические проверки функций и ограничений целостности. Проверенные расчёты совпали с ожидаемыми значениями; недопустимые данные отклонялись.

3.7.3 Проверка прикладных сценариев, прав и кэша

Для прикладного уровня определены сценарии из таблицы 3.6. Их ожидаемые результаты следуют из проектируемого интерфейса, матрицы полномочий и алгоритма кэширования. Проверка SQL-функций и ограничений сама по себе не подтверждает работу пользовательского интерфейса или Redis.

Таблица 3.6 — Сценарии проверки прикладного уровня

Сценарий	Критерий корректности
Выбор существующего участника	Показатели совпадают с прямым вызовом функции БД
Неверный формат идентификатора	Ошибка ввода до выполнения расчёта
Неизвестный участник	Сообщение об отсутствии записи
Изменение результата аналитиком	Отказ в доступе; данные не изменены
Запись допустимого результата оператором	Успешная фиксация транзакции
Недопустимая строка в протоколе	Откат всех изменений данной операции
Первое и повторное чтение статистики	Одинаковые данные при промахе и попадании в кэш
Истечение 60 с	Повторное вычисление и заполнение кэша
Изменение данных и смена поколения	Новые запросы не используют ключи прежнего поколения
Откат изменения результатов	Поколение кэша не меняется
Недоступность Redis	Результат получен из PostgreSQL
Параллельное чтение и исправление	Кэш не считается гарантией немедленной актуальности; прямой запрос возвращает состояние БД

Вывод

Описаны сущности PostgreSQL, ограничения целостности и реализация расчётных функций. Определены SQL-полномочия ролей и операции приложения доступа. Для Redis приведены формат ключей, срок жизни результатов и порядок обновления кэша. Расчёты и ограничения проверены автоматическими тестами; для прикладного уровня сформулированы отдельные сценарии тестирования.

4 Исследовательский раздел

4.1 Технические характеристики

Технические характеристики устройства, на котором выполнялось тестирование:

— ноутбук MacBook Air M3 2024 [9]

— операционная система – MacOS Tahoe 26.6.2 [10];

— оперативная память – 8 Гбайт;

— процессор – Apple M3.

Сервер PostgreSQL и клиент находились на одном компьютере. Измерения проводились в отдельной базе с таблицами и расчётными функциями, описанными в технологическом разделе. Перед измерениями повторно выполнены функциональные проверки; они завершились успешно.

При настройке сервера отключены JIT и параллельные работники отдельного запроса. Параллельные клиентские соединения при этом разрешены. Фоновые процессы операционной системы не отключались и могли влиять на результаты. Значение `effective_cache_size` является параметром оценки планировщика, а не выделенным объёмом памяти.

Генератор создаёт один чемпионат, один класс, одну трассу, одну модель автомобиля, 12 команд и 24 заявки с одним гонщиком в каждой. У команды две машины; в каждой гонке формируются 24 финиша и 24 квалификации. Все заезды имеют тип 2, длительности 6, 8 и 24 часа чередуются. Позиции задаются циклическими сдвигами номеров участников. Такой набор воспроизводим и задействует все коэффициенты начисления очков.

Данные являются синтетическим наполнением базы, а значения времени получены при реальном выполнении SQL-функций на этом наполнении. Большое число гонок в одном чемпионате используется для создания нагрузки и не описывает календарь реального сезона.

После каждой загрузки выполнялась команда `VACUUM ANALYZE`. Индексы не изменялись между прогонами. Перед измерениями для функции выполнялись 10 прогревочных запросов на клиента. Кэш ОС и PostgreSQL принудительно не очищался, поэтому результаты относятся к режиму после прогрева.

4.2 Зависимость задержки от числа гонок

Исследованы функции `calculatepersonalpoints`, `calculatedriverstats` и `calculateteamstats`. Их параметр равен 1. Число гонок N принимает значения 20, 100, 500, 1000 и 5000; число записей каждой таблицы результатов составляет

$$M_{\text{finish}} = M_{\text{qualifying}} = 24N. \tag{4.1}$$

Таким образом, максимальный набор содержит 120 000 финишей и столько же квалификаций. Изменяется весь объём связанных фактов, а не только число строк таблицы гонок.

На каждом размере выполнялись три серии по 100 запросов каждой функции, один клиент и один поток `pgbench`. Порядок серий функций перемешивался с фиксированным начальным значением генератора случайных чисел. Всего выполнено 45 измерительных серий, то есть 4500 запросов без учёта прогрева.

Для серии j инструмент возвращает среднюю задержку $\bar{t}_j$. Для каждого сочетания параметров вычисляется среднее арифметическое трёх серий:

$$\bar{t} = \frac{\bar{t}_1 + \bar{t}_2 + \bar{t}_3}{3}. \tag{4.2}$$

Среднее относится к средним задержкам серий, а не к объединённому набору отдельных запросов.

Результаты представлены в таблице 4.1. Все серии обработали предусмотренные 100 транзакций. На рисунке 4.1 приведены средние значения. На обеих осях использованы линейные шкалы с равным шагом делений.

Таблица 4.1 — Задержка расчётных запросов при увеличении объёма, мс

Гонки N	Строк в `finish`	Очки заявок	Гонщик	Команда
20	480	0,454	0,652	0,690
100	2 400	1,587	1,859	1,980
500	12 000	6,885	7,709	8,471
1000	24 000	13,903	15,674	17,880
5000	120 000	67,952	75,335	81,547

В ячейке указана средняя задержка трёх серий.

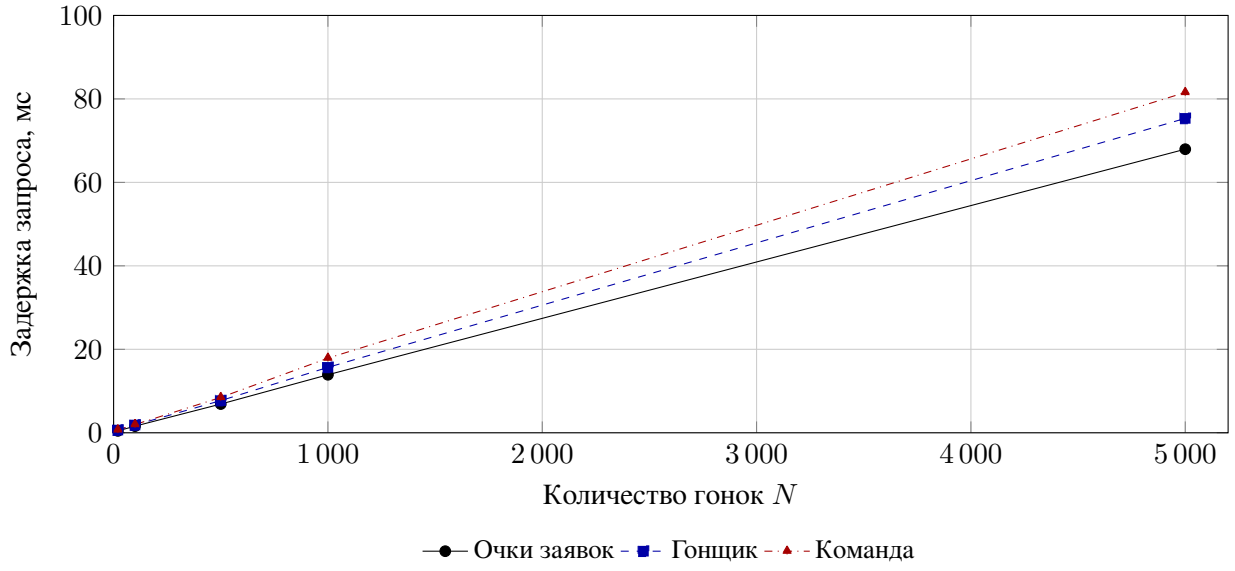


Рисунок 4.1 — Задержка в зависимости от числа гонок

При увеличении числа гонок с 20 до 5000, то есть в 250 раз, средняя задержка функции очков выросла с $0{,}454$ до $67{,}952$ мс, функции гонщика — с $0{,}652$ до $75{,}335$ мс, функции команды

— с $0{,}690$ до $81{,}547$ мс. Относительный рост составил соответственно $149{,}8$, $115{,}6$ и $118{,}1$ раза. Меньший по сравнению с ростом числа записей коэффициент объясним наличием постоянных затрат запроса на малом наборе, но сам по себе не доказывает конкретную асимптотическую сложность.

На участке от 1000 до 5000 гонок объём увеличился в пять раз, а средние задержки — в $4{,}89$, $4{,}81$ и $4{,}56$ раза соответственно. В этом диапазоне наблюдаемая зависимость близка к линейной. На каждом размере статистика команды выполнялась дольше статистики гонщика и простого расчёта очков, что согласуется с дополнительными агрегатами и построением командной классификации. Выделение вклада каждой операции потребовало бы отдельного исследования планов выполнения.

4.3 Зависимость задержки от конкурентной нагрузки

Во втором исследовании использован набор из 1000 гонок и 24 000 финишей. Все клиенты выполняли запрос `SELECT * FROM calculatepersonalpoints(1)`. Число клиентов C изменялось по значениям 1, 2, 4, 8, 16 и 32. Каждый клиент передавал следующий запрос только после получения ответа на предыдущий.

Это закрытая модель нагрузки: число клиентов определяет верхнюю границу числа запросов в работе, а интенсивность поступления зависит от скорости ответов. Она не моделирует неограниченную внешнюю очередь. Одновременные изменения данных не выполнялись, поэтому оценивалась конкуренция читающих запросов за вычислительные ресурсы.

Для каждого значения C выполнены три серии по 30 секунд после прогрева. Число рабочих потоков клиента равно $\min(C, 8)$. Порядок сочетаний числа клиентов и номера серии перемешивался с фиксированным начальным значением генератора. Продолжительность измерительных интервалов — 540 секунд суммарно, без загрузки, прогрева и установления соединений.

Задержка обобщается по формуле (4.2). Коэффициент $K(C) = \bar{t}(C)/\bar{t}(1)$ показывает изменение средней задержки относительно одноклиентского режима. Значения базового прогона относятся ко второму исследованию; подмена их результатом первого исследования не выполнялась.

Таблица 4.2 — Задержка запроса при конкурентной нагрузке

Клиенты C	Задержка, мс	$K(C)$
1	14,012	1,00
2	13,687	0,98
4	18,637	1,33
8	30,087	2,15
16	58,573	4,18
32	111,716	7,97

Задержка: среднее трёх серий.

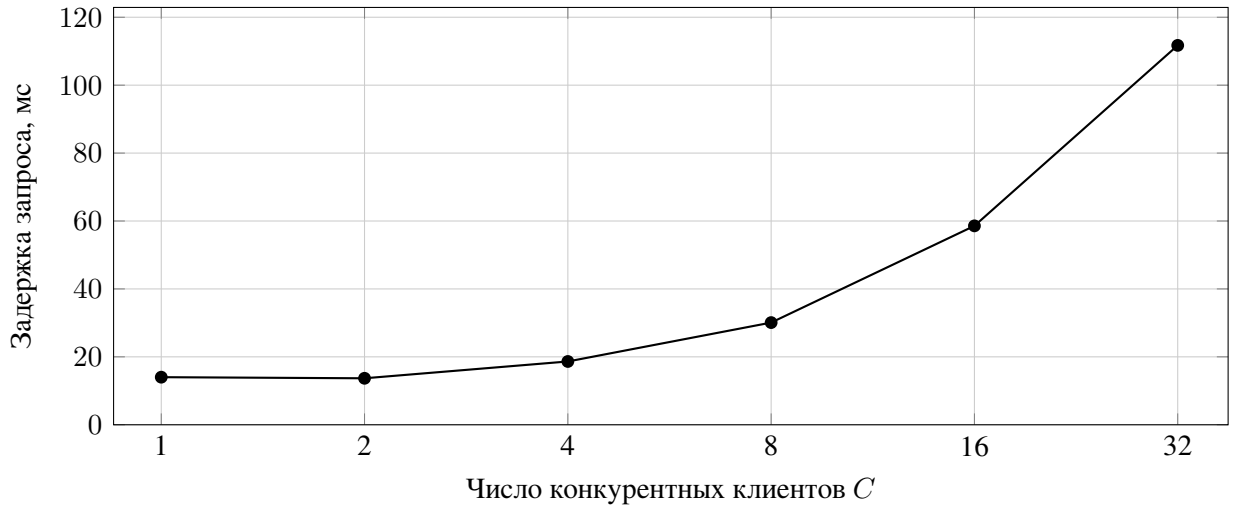


Рисунок 4.2 — Задержка запроса при увеличении числа конкурентных клиентов

На рисунке 4.2 показаны средние значения результатов трёх серий. По горизонтали используется логарифмическая шкала с основанием 2, поскольку число клиентов удваивается.

Средняя задержка одного запроса выросла с $14{,}012$ мс при одном клиенте до $111{,}716$ мс при 32 клиентах, то есть в $7{,}97$ раза. Следовательно, при исследованном объёме данных увеличение числа одновременных клиентов существенно увеличивает время ответа.

При переходе от одного к двум клиентам средняя задержка немного уменьшилась: с $14{,}012$ до $13{,}687$ мс. Разница невелика, поэтому по трём сериям нельзя считать её устойчивым эффектом ускорения отдельного запроса.

При переходе от 4 к 32 клиентам средняя задержка увеличилась с $18{,}637$ до $111{,}716$ мс, то есть примерно в шесть раз. Такой рост согласуется с конкуренцией запросов за вычислительные ресурсы стенда, но без аппаратного профилирования нельзя разделить вклад процессора, памяти и планирования процессов.

При увеличении числа клиентов с 16 до 32 средняя задержка выросла с $58{,}573$ до $111{,}716$ мс, то есть в $1{,}91$ раза. Все серии завершились без ошибок выполнения.

4.4 Обработка результатов и ограничения исследования

При обработке результатов использованы 45 серий первого исследования и 18 серий второго. Для каждого сочетания параметров вычислено среднее арифметическое. Таблицы и графики построены по одним и тем же сводным данным. Округление применяется только при отображении числовых значений.

Воспроизводимость исследования обеспечивается фиксированными параметрами набора данных, порядком прогрева и числом повторов. При повторении необходимо сохранять конфигурацию PostgreSQL, состав индексов и режим нагрузки, указанные в описании исследований.

Полученные значения имеют ограничения применимости. Один чемпионат и один класс не характеризуют базу с большим числом исторических сезонов; функции статистики обращаются к чемпионатам при вычислении позиций. Синтетические позиции не воспроизводят

распределение реальных выступлений. Локальный обмен не учитывает сетевую задержку удалённого клиента. Прогрев не описывает первый запрос после запуска сервера. Отсутствие записывающих транзакций исключает оценку конфликтов обновления.

Трёх серий недостаточно для точной оценки статистической неопределённости или универсального порога производительности. Во время испытаний не выполнялись аппаратный контроль температуры и изоляция от всех фоновых процессов. Для решения о промышленной нагрузке необходимы более длительные измерения на целевом оборудовании с характерными пользовательскими запросами.

Вывод

Проведены два исследования на одном стенде, включающие 63 измерительные серии. При росте числа гонок от 20 до 5000 средняя задержка расчёта очков увеличилась с $0{,}454$ до $67{,}952$ мс; в диапазоне 1000–5000 гонок рост задержки всех трёх функций близок к росту числа записей. Увеличение числа клиентов с 1 до 32 повысило среднюю задержку запроса с $14{,}012$ до $111{,}716$ мс, или в $7{,}97$ раза. Полученные зависимости относятся к используемому набору данных, конфигурации и режиму после прогрева.

ЗАКЛЮЧЕНИЕ

В работе рассмотрена задача хранения и обработки результатов гоночных заездов мировых чемпионатов. Сопоставлены существующие решения, уточнены требования и ограничения предметной области. По результатам сравнения моделей хранения выбрана реляционная модель. Разделение справочных сущностей и результатов позволяет использовать одни и те же сведения об участниках при обработке нескольких заездов.

Спроектирована структура из 13 отношений и проверено её соответствие третьей нормальной форме. Определены первичные и внешние ключи, ограничения допустимых значений и индексы для основных условий поиска. Формализованы расчёт очков, статистика гонщика и статистика команды. Уточнены различия между числом гонок и числом финишей, между результатом экипажа и личной классификацией, между промежуточным лидерством и официальным титулом.

Средствами PostgreSQL реализованы сущности, функции расчёта очков и статистики, запросы получения протоколов. Функция начисления очков универсальна для чемпионатов, хранящихся в БД. Уникальность результатов обеспечивается составными первичными ключами, допустимость значений — доменными проверками. Автоматические тесты подтвердили ожидаемые расчёты и реакции на недопустимые данные в пределах тестового набора.

Определены полномочия владельца, оператора и аналитика. Описаны операции приложения доступа и схема внешнего кэширования в Redis с ограниченным сроком жизни результатов и сменой поколения после изменения данных. Для проверки прав, интерфейса и согласованности кэша сформулированы отдельные тестовые сценарии.

Проведены два исследования на одном стенде, включающие 63 измерительные серии. При росте числа гонок от 20 до 5000 средняя задержка расчёта очков увеличилась с $0{,}454$ до $67{,}952$ мс; в диапазоне 1000–5000 гонок рост задержки всех трёх функций близок к росту числа записей. Увеличение числа клиентов с 1 до 32 повысило среднюю задержку запроса с $14{,}012$ до $111{,}716$ мс, или в $7{,}97$ раза. Полученные зависимости относятся к используемому набору данных, конфигурации и режиму после прогрева.

Дальнейшее развитие базы может включать хранение статуса результата, привязку состава экипажа к отдельному заезду и полноценный личный зачёт при переходе гонщика между заявками. Эти изменения позволят точнее отражать состав выступлений и итоговые классификации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Formula 1. 2025 Race Results [Электронный ресурс]. URL: https://www.formula1.com/en/results/2025/races (дата обращения: 25.05.2026).
2. FIA World Endurance Championship. 2025 Season: Results and Standings [Электронный ресурс]. URL: https://www.fiawec.com/en/season/2025 (дата обращения: 30.05.2026).
3. IMSA. WeatherTech SportsCar Championship Results [Электронный ресурс]. URL: https://www.imsa.com/weathertech/results/ (дата обращения: 30.05.2026).
4. IMSA. WeatherTech SportsCar Championship Standings [Электронный ресурс]. URL: https://www.imsa.com/weathertech/standings/ (дата обращения: 30.05.2026).
5. Jolpica F1. API Documentation [Электронный ресурс]. URL: https://github.com/jolpica/jolpica-f1/blob/main/docs/README.md (дата обращения: 25.05.2026).
6. PostgreSQL. Documentation [Электронный ресурс]. URL: https://www.postgresql.org/docs/ (дата обращения: 15.05.2026).
7. Redis Documentation. Redis cache-aside [Электронный ресурс]. URL: https://redis.io/docs/latest/develop/use-cases/cache-aside/ (дата обращения: 27.05.2026).
8. Redis Documentation. SET [Электронный ресурс]. URL: https://redis.io/docs/latest/commands/set/ (дата обращения: 27.05.2026).
9. MacBook Air (13 дюймов, M3, 2024 г.) [Электронный ресурс]. Режим доступа: https://support.apple.com/ru-ru/docs/mac/301019 (дата обращения: 15.05.2026)
10. macOS Tahoe 26.0 [Электронный ресурс]. Режим доступа: https://www.apple.com/os/macos/ (дата обращения: 15.05.2026)